Experiment 14: Unfolded RNN Architecture Visualization

Question

Problem Statement: A new ML engineer at a speech-analytics firm finds the recurrent loop hard to follow. The team lead asks for a diagram showing the RNN unrolled over time. A 5-word input sentence passed through a SimpleRNN with a 3-unit hidden state. The objective is to visualise an RNN unfolded across time steps and trace the hidden-state values. Use TensorFlow, Keras, NetworkX, Matplotlib.

Tasks:

1. Build a SimpleRNN in Keras and extract its input, recurrent and output weights.
2. Compute hidden states for each time step manually and confirm them against Keras.
3. Draw the unfolded network as a graph with NetworkX (nodes $x_1…x_5$, $h_1…h_5$, $y_1…y_5$).
4. Label edges with shared weights W_xh, W_hh and W_hy.
5. Plot the hidden-state values across time steps.

Code

```
"""Experiment 14: Unfolded RNN Architecture Visualization"""
import numpy as np
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

np.random.seed(1); tf.random.set_seed(1)

words = ["I", "love", "deep", "learning", "models"]       # 5-word sentence
T, EMB, UNITS, OUT = 5, 4, 3, 2
emb = np.random.randn(T, EMB)                              # a fixed 4-D vector for each word
X = emb[np.newaxis].astype("float32")                      # shape (1, 5, 4)

# ---------------- Task 1: build SimpleRNN and extract weights ----------------
inp = keras.Input((T, EMB))
rnn = layers.SimpleRNN(UNITS, activation="tanh", return_sequences=True, name="rnn")
h_seq = rnn(inp)
y_seq = layers.Dense(OUT, name="out")(h_seq)               # y_t = W_hy h_t + b_y at every step
model = keras.Model(inp, y_seq)
rnn_model = keras.Model(inp, h_seq)

W_xh, W_hh, b_h = rnn.get_weights()                        # input, recurrent kernels and bias
W_hy, b_y = model.get_layer("out").get_weights()
print("W_xh (input -> hidden):", W_xh.shape); print(np.round(W_xh, 3))
print("W_hh (hidden -> hidden):", W_hh.shape); print(np.round(W_hh, 3))
print("W_hy (hidden -> output):", W_hy.shape); print(np.round(W_hy, 3))

# ---------------- Task 2: manual hidden states vs Keras ----------------
h = np.zeros(UNITS)
manual_h, manual_y = [], []
for t in range(T):
    h = np.tanh(X[0, t] @ W_xh + h @ W_hh + b_h)           # h_t = tanh(x_t W_xh + h_{t-1} W_hh + b)
    manual_h.append(h); manual_y.append(h @ W_hy + b_y)    # y_t = h_t W_hy + b_y
manual_h, manual_y = np.array(manual_h), np.array(manual_y)
keras_h = rnn_model.predict(X, verbose=0)[0]
keras_y = model.predict(X, verbose=0)[0]

table = pd.DataFrame(manual_h, index=[f"h{t+1} ({w})" for t, w in enumerate(words)],
                     columns=[f"unit{u+1}" for u in range(UNITS)])
print("\nManual hidden states:\n", table.round(4))
print("Matches Keras hidden states:", np.allclose(manual_h, keras_h, atol=1e-5))
print("Matches Keras outputs      :", np.allclose(manual_y, keras_y, atol=1e-5))

# ---------------- Task 3 & 4: unfolded graph with NetworkX ----------------
G = nx.DiGraph()
pos = {}
```

```
for t in range(1, T + 1):
    G.add_node(f"x{t}"); G.add_node(f"h{t}"); G.add_node(f"y{t}")
    pos[f"x{t}"], pos[f"h{t}"], pos[f"y{t}"] = (t * 2, 0), (t * 2, 1.5), (t * 2, 3)
    G.add_edge(f"x{t}", f"h{t}", label="W_xh")
    G.add_edge(f"h{t}", f"y{t}", label="W_hy")
    if t > 1:
        G.add_edge(f"h{t-1}", f"h{t}", label="W_hh")

plt.figure(figsize=(14, 6))
colors = ["#9ecae1" if n[0] == "x" else "#a1d99b" if n[0] == "h" else "#fdae6b" for n in G.nodes]
nx.draw_networkx_nodes(G, pos, node_color=colors, node_size=1400, edgecolors="k")
nx.draw_networkx_labels(G, pos, font_size=11)
nx.draw_networkx_edges(G, pos, arrowsize=18, node_size=1400)
nx.draw_networkx_edge_labels(G, pos, edge_labels=nx.get_edge_attributes(G, "label"), font_size=9,
                             font_color="darkred", label_pos=0.5)
for t, w in enumerate(words, start=1):
    plt.text(t * 2, -0.45, f'"{w}"', ha="center", fontsize=10)
plt.title("RNN unfolded over 5 time steps (the same W_xh, W_hh, W_hy are shared at every step)")
plt.axis("off"); plt.tight_layout(); plt.show()

# ---------------- Task 5: hidden-state values across time ----------------
fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
for u in range(UNITS):
    ax[0].plot(range(1, T + 1), manual_h[:, u], "o-", label=f"unit {u+1}")
ax[0].set_xticks(range(1, T + 1)); ax[0].set_xlabel("Time step"); ax[0].set_ylabel("Hidden state value")
ax[0].set_title("Hidden-state trajectories"); ax[0].legend(); ax[0].grid(alpha=0.3)
im = ax[1].imshow(manual_h.T, cmap="coolwarm", vmin=-1, vmax=1, aspect="auto")
ax[1].set_xticks(range(T)); ax[1].set_xticklabels([f"h{t+1}" for t in range(T)])
ax[1].set_yticks(range(UNITS)); ax[1].set_yticklabels([f"unit {u+1}" for u in range(UNITS)])
for t in range(T):
    for u in range(UNITS):
        ax[1].text(t, u, f"{manual_h[t, u]:.2f}", ha="center", va="center")
ax[1].set_title("Hidden-state heatmap"); plt.colorbar(im, ax=ax[1])
plt.tight_layout(); plt.show()
```

Output

```
W_xh (input -> hidden): (4, 3)
[[-0.363  0.512  0.679]
 [-0.485  0.65   0.292]
 [-0.682 -0.325  0.537]
 [-0.338  0.34  -0.568]]
W_hh (hidden -> hidden): (3, 3)
[[-0.011 -0.722  0.692]
 [ 0.559 -0.579 -0.594]
 [-0.829 -0.38  -0.41 ]]
W_hy (hidden -> output): (3, 2)
[[-0.287  0.386]
 [-0.917 -0.791]
 [ 0.642 -0.015]]

Manual hidden states:
                 unit1   unit2   unit3
h1 (I)         0.3633  0.2557  0.6482
h2 (love)     -0.7116 -0.5098  0.7010
h3 (deep)     -0.9865 -0.3675  0.9540
h4 (learning) -0.8057 -0.5737  0.1022
h5 (models)   -0.2068  0.3799 -0.7357
Matches Keras hidden states: True
Matches Keras outputs      : True
```

